

Context Matters: Expanding the Potential of eBPF

Carlos Machado, Sebastião Amaro*, Miguel Matos*, João Paulo, Tânia Esteves
INESC TEC & U. Minho *IST Lisbon & INESC-ID

Abstract

The extended Berkeley Packet Filter (eBPF) has become a powerful platform for extending kernel functionality at run-time, yet eBPF programs lack visibility into end-to-end I/O request intent across the kernel stack, limiting a broad class of storage optimizations. We propose extending eBPF with first-class support for context propagation in the I/O stack. We motivate this need, illustrate use cases that demonstrate the opportunities it enables, and discuss possible design directions and the key challenges they pose.

CCS Concepts: • Software and its engineering → Operating systems.

Keywords: eBPF, Context Propagation, Kernel Extension

ACM Reference Format:

Carlos Machado, Sebastião Amaro*, Miguel Matos*, João Paulo, Tânia Esteves. 2026. Context Matters: Expanding the Potential of eBPF. In *4th Workshop on eBPF and Kernel Extensions (eBPF'26)*, September 29–October 02, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3837779.3838163>

1 Introduction and Problem Statement

eBPF is increasingly shifting from *observing* kernel behavior to *acting* on it, directly shaping CPU [10, 13], storage [17, 30] and memory management [4, 32]. This creates an opportunity to redesign and safely port traditional optimizations (e.g., caching, scheduling) that are implemented directly at the application level (e.g., databases, key-value stores) into the kernel, promoting their generality and portability.

We focus on storage optimizations and show that eBPF lacks the ability to propagate application-level context along the kernel execution path of a storage I/O request. Modern data-intensive applications have workloads rich in semantic structure invisible to the kernel (e.g., databases distinguish latency-critical foreground writes from housekeeping tasks [1, 3]). Exploiting this context to drive kernel decisions (e.g., scheduling, rate-limiting) requires awareness, but this information is lost as a request descends the kernel I/O stack.

eBPF maps are not enough. Maps partially address this by enabling data exchange between user and kernel spaces, and by allowing metadata to be attached to tasks and kernel

objects. Yet, these fall short of propagating context along the I/O stack. First, the OS page cache asynchronously flushes buffered data, decoupling write requests from the issuing threads, as writeback runs later via kernel threads (*kworker*) on different CPU/cores, making task-keyed maps inaccessible. Second, a single thread may serve I/O requests with different purposes (e.g., different clients or transactions), indistinguishable in kernel. The same holds for kernel mechanisms that already carry some context (e.g., cgroup association, I/O priority): they operate at a coarse level (task/cgroup) rather than per-request and expose a limited vocabulary (e.g., priority classes) instead of application-level information.

Motivation example. Figure 1 illustrates such problems by tracking the kernel request flow of two concurrent application threads performing sequential 4 KiB writes to the same file, across three phases: (P1) each thread concurrently issues five writes; (P2) both threads sleep long enough for the kernel to flush dirty pages at the page cache; and (P3) each thread issues one final write before thread B calls `fsync`.

eBPF can trace write operations across the system call (`syscall`), Virtual File System (VFS), `ext4`, and block I/O (BIO) layers with thread-level context (thread identifier). However, once writes are buffered in the page cache, they may be merged and flushed asynchronously by *kworker* threads. One can attempt to correlate BIO requests (`block_rq_issue`) with the write operations that originated them based on request sizes (e.g., ten consecutive 4 KiB writes may produce a single 40 KiB BIO request). Yet, as illustrated in the figure, multiple unrelated 40 KiB BIO requests may be issued by *kworkers* (e.g., the request handled by *kworker* 166 is unrelated to our application), making this correlation unreliable.

To illustrate how context propagation could be useful, we run a simple experiment. Write buffers are hashed in all layers (`syscall`, VFS, `ext4`) and matched in the BIO layer by a custom kernel module. Different contents are assigned unique colors in Figure 1. We can see that writes from both application threads are merged across layers, and by phase P3 the final `block_rq_issue`, issued by thread B (21719), carries data from both threads in a single BIO operation. This correlation is only visible due to our kernel module, as eBPF cannot directly access the data in `block_rq_issue` events.

The question raised in this paper is: *Can eBPF provide context propagation without user-defined custom kernel modules?*

Proposal. Our goal is to extend eBPF to attach application context at strategic points in the I/O path and propagate it with the request throughout the kernel stack. A tag, or set of tags (e.g., request identifier, priority class, or arbitrary tag) would flow through the VFS, filesystem, and BIO layer



This work is licensed under a Creative Commons Attribution 4.0 International License.

eBPF'26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2911-9/2026/09

<https://doi.org/10.1145/3837779.3838163>

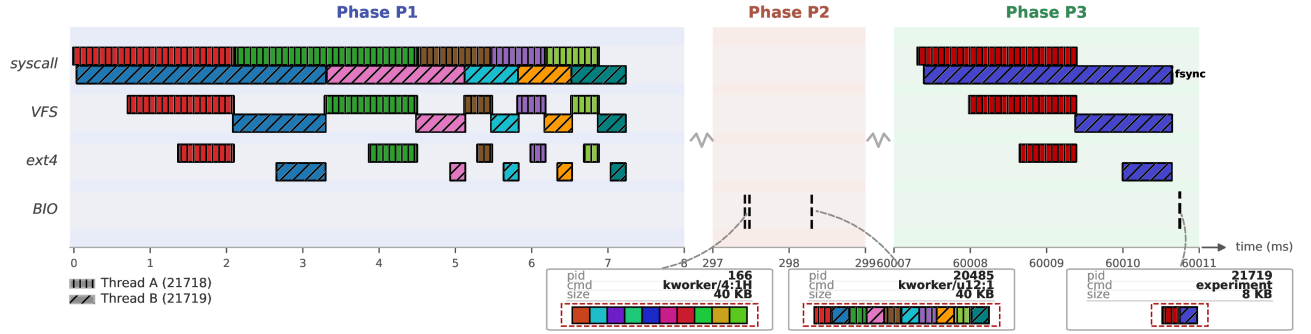


Figure 1. Correlation of write events across kernel layers using content hashes (same color indicates the same hash). Most of the information is collected using eBPF, while BIO segment contents (dashed red line) are obtained via a custom kernel module.

without being lost across asynchronous boundaries such as page-cache writeback and kworker dispatch. eBPF programs at any layer could read and act on this tag without manual map management or per-layer reconstruction. This would unlock several eBPF-based optimizations and use-cases.

Observability and data provenance. Distributed tracing solutions propagate context across component/node boundaries, but leave the kernel I/O flow opaque [6, 19, 28, 31]. Systems addressing this gap only provide partial visibility, either correlating user and kernel events offline [7, 22], or propagating kernel-internal IDs with no application-level semantics [16]. Application-level context would unlock root-cause analysis and reproduction of I/O anomalies by pinpointing exactly the responsible kernel layer(s) and application request(s) in an end-to-end fashion [14, 18, 21, 24, 26].

Application-aware I/O scheduling. In-kernel I/O schedulers operate on coarse-grained contexts like PID and request size, unaware of application semantics [3, 8, 11]. Context tags carried through BIO operations would let schedulers make priority and fairness decisions at the granularity of individual requests or tenants, improving performance and reducing I/O interference [20, 27].

Specialized I/O optimizations. The best tuning parameters for in-kernel optimizations may depend on the application workflow [5, 9, 12, 15, 20, 23, 32]. For instance, scans from key-value stores benefit from read prefetching, but random accesses may not. Context propagation would allow stamping requests at their origin and tuning these on a per-application/request basis.

2 Design Directions and Challenges

Extending eBPF with context propagation requires three fundamental capabilities: (i) *attaching* context to a request at its origin; (ii) *propagating* it to and across kernel layers, surviving asynchronous boundaries; and (iii) *extracting* it at any layer. We identify two possible directions to explore.

Modifying eBPF and the kernel. The most direct approach is to extend eBPF and the kernel to natively support context tags. For synchronous layer transitions, existing

eBPF mechanisms such as task storage maps or stack traces could help propagate context, as in [25]. Asynchronous boundaries need more exploratory approaches (e.g., wrapping kworker dispatch to carry the originating context tag [28]), potentially extending kernel structures with dedicated fields [19, 29] and new kfuncs. This enables richer semantics, but sacrifices portability and maintainability. Yet, sched_ext is an example of such a path, whose kernel modifications are now part of mainline Linux [10].

A transparent middleware layer. A less intrusive alternative is to implement context propagation as a middleware service that eBPF programs can interact with through a stable API, without modifying the kernel’s core data structures. eBPF programs would call into the middleware to attach context at the syscall boundary and retrieve it at lower layers, with the middleware maintaining the cross-layer association internally. This approach is more portable, but faces a harder technical challenge: reliably identifying the same request across kernel layers and asynchronous boundaries without a native kernel carrier, since existing identifiers (e.g., PIDs, inode numbers) are insufficient, as shown earlier.

Both directions must also address *safety* (compliance with the eBPF verifier and kernel integrity), *performance overhead* (over the critical I/O path) and *semantic ambiguity* (a single user-space write may translate into multiple kernel write requests (e.g., due to journaling or copy-on-write [2])).

Although ambitious, we believe extending eBPF with context propagation will enable kernel optimizations now confined to less portable, less generic user-space workarounds.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback. This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., under the support UID/50014/2025 (<https://doi.org/10.54499/UID/50014/2025>) (Carlos Machado).

References

- [1] 2026. Postgresql: WAL Configuration. Retrieved June, 2026 from <https://www.postgresql.org/docs/current/wal-configuration.html>
- [2] Remzi H Arpaci-Dusseau and Andrea C Arpaci-Dusseau. 2018. *Operating systems: Three easy pieces*. Vol. 1. Arpaci-Dusseau Books, LLC Madison, WI, USA.
- [3] Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhramoorthi, and Diego Didona. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *Annual Technical Conference*. USENIX, 753–766. <https://www.usenix.org/conference/atc19/presentation/balmau>
- [4] Xuechun Cao, Shaurya Patel, Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. 2024. FetchBPF: Customizable Prefetching Policies in Linux with eBPF. In *Annual Technical Conference*. USENIX, 369–378. <https://www.usenix.org/conference/atc24/presentation/cao>
- [5] Vijay Chidambaram, Thanumalayan Sankaranarayanan Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2013. Optimistic crash consistency. In *Symposium on Operating Systems Principles*. ACM, 228–243. doi:10.1145/2517349.2522726
- [6] CNCF. 2016. Jaeger: Open Source, End-to-End Distributed Tracing. <https://www.jaegertracing.io>
- [7] Tânia Esteves, Francisco Neves, Rui Oliveira, and João Paulo. 2021. CAT: content-aware tracing and analysis for distributed systems. In *International Middleware Conference*. ACM, 223–235. doi:10.1145/3464298.3493396
- [8] Facebook / Meta Kernel Team. 2020. IOCost: I/O Cost-Based Weight-QoS Controller. Linux kernel v5.4. <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>
- [9] Zhiyuan Guo, Zijian He, and Yiyang Zhang. 2023. Mira: A Program-Behavior-Guided Far Memory System. In *Symposium on Operating Systems Principles*. ACM, 692–708. doi:10.1145/3600006.3613157
- [10] Tejun Heo and David Vernet. 2024. sched_ext: eBPF Extensible Scheduler Class. Linux kernel v6.12. Retrieved June, 2026 from <https://github.com/sched-ext/scx>
- [11] Jaehyun Hwang, Midhul Vuppapapati, Simon Peter, and Rachit Agarwal. 2021. Rearchitecting Linux Storage Stack for μ s Latency and High Throughput. In *Symposium on Operating Systems Design and Implementation*. USENIX Association, 113–128. <https://www.usenix.org/conference/osdi21/presentation/hwang>
- [12] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. CLOCK-Pro: An Effective Improvement of the CLOCK Replacement. In *Annual Technical Conference*. USENIX, 323–336. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>
- [13] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. 2021. Syrup: User-Defined Scheduling Across the Stack. In *Symposium on Operating Systems Principles*. ACM, 605–620. doi:10.1145/3477132.3483548
- [14] Mitsumasa Kondo. 2026. SteelDB: Diagnosing Kernel-Space Bottlenecks in Cloud OLTP Databases. *arXiv preprint arXiv:2603.29052* (2026). <https://arxiv.org/abs/2603.29052>
- [15] Changman Lee, Dongho Sim, Jooyoung Hwang, and Sangyeon Cho. 2015. F2FS: A New File System for Flash Storage. In *Conference on File and Storage Technologies*. USENIX, 273–286. <https://www.usenix.org/conference/fast15/technical-sessions/presentation/lee>
- [16] Ganguk Lee, Yeaseul Park, Jeongseob Ahn, and Youngjin Kwon. 2019. Slicing the io execution with relaytracer. *arXiv preprint arXiv:1906.07124* (2019).
- [17] Chuandong Li, Ran Yi, Zonghao Zhang, Jing Liu, Changwoo Min, Jie Zhang, Yingwei Luo, Xiaolin Wang, Zhenlin Wang, and Diyu Zhou. 2025. Aeolia: A Fast and Secure Userspace Interrupt-Based Storage Stack. In *Symposium on Operating Systems Principles*. ACM, 479–495. doi:10.1145/3731569.3764816
- [18] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. 2014. Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency. In *Symposium on Cloud Computing*. ACM, 1–14. doi:10.1145/2670979.2670988
- [19] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. 2018. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. *ACM Trans. Comput. Syst.* 35, 4, Article 11 (2018), 28 pages. doi:10.1145/3208104
- [20] Ricardo Macedo, Yusuke Tanimura, Jason Haga, Vijay Chidambaram, José Pereira, and João Paulo. 2022. PAIO: General, Portable I/O Optimizations With Minor Application Modifications. In *Conference on File and Storage Technologies*. USENIX, 413–428. <https://www.usenix.org/conference/fast22/presentation/macedo>
- [21] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnappalli, Pandian Raju, and Vijay Chidambaram. 2019. CrashMonkey and ACE: Systematically Testing File-System Crash Consistency. *ACM Trans. Storage* 15, 2, Article 14 (2019), 34 pages. doi:10.1145/3320275
- [22] Mohammed Islam Naas, François Trahay, Alexis Colin, Pierre Olivier, Stéphane Rubini, Frank Singhoff, and Jalil Boukhobza. 2021. EZIO-Tracer: unifying kernel and user space I/O tracing for data-intensive applications. In *Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*. ACM, 1–11. doi:10.1145/3439839.3458731
- [23] R. H. Patterson, G. A. Gibson, E. Ginting, D. Stodolsky, and J. Zelenka. 1995. Informed prefetching and caching. In *Symposium on Operating Systems Principles*. ACM, 79–95. doi:10.1145/224056.224064
- [24] Thanumalayan Sankaranarayanan Pillai, Vijay Chidambaram, Ramnathan Alagappan, Samer Al-Kiswani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2014. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Operating Systems Design and Implementation*. USENIX, 433–448. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai>
- [25] Polar Signals. 2024. Parca: Open Source Infrastructure-Wide Continuous Profiling. <https://www.parca.dev>
- [26] Maria Ramos, João Azevedo, Kyle Kingsbury, José Pereira, Tânia Esteves, Ricardo Macedo, and Flávio De Oliveira Silva. 2024. When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection. *Proc. VLDB Endow.* 17, 11 (2024), 3017–3030. doi:10.14778/3681954.3681980
- [27] Philipe S. Schiavo, João P. S. Milanezi, Moisés R. N. Ribeiro, Victor M. G. Martinez, João Henrique Corrêa, José Marcos Nogueira, Fernando Frota Redigolo, Tereza C. Carvalho, and Flávio De Oliveira Silva. 2026. Causal Inference for Quantifying Noisy Neighbor Effects in Multi-Tenant Cloud Environments. In *International Conference on Communications*. IEEE, 1–7. doi:10.1109/ICC59461.2026.11587133
- [28] Benjamin H Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaskan, and Chandan Shanbhag. 2010. Dapper, a large-scale distributed systems tracing infrastructure. <https://research.google.com/archive/papers/dapper-2010-1.pdf>
- [29] Suli Yang, Tyler Harter, Nishant Agrawal, Salini Selvaraj Kowsalya, Anand Krishnamurthy, Samer Al-Kiswani, Rini T. Kaushik, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2015. Split-level I/O scheduling. In *Symposium on Operating Systems Principles*. ACM, 474–489. doi:10.1145/2815400.2815421
- [30] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Symposium on Operating Systems Design and Implementation*. USENIX, 375–393. <https://www.usenix.org/conference/osdi22/presentation/zhong>
- [31] Zipkin Contributors. 2012. Zipkin. <https://zipkin.io>
- [32] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. cache_ext: Customizing the Page Cache with eBPF. In *Symposium on Operating Systems Principles*. ACM, 462–478. doi:10.1145/3731569.3764820